Exercise 10

Elisa Schöpf, Leon Krieger, Nikola Adzic

May 31, 2026

Contents

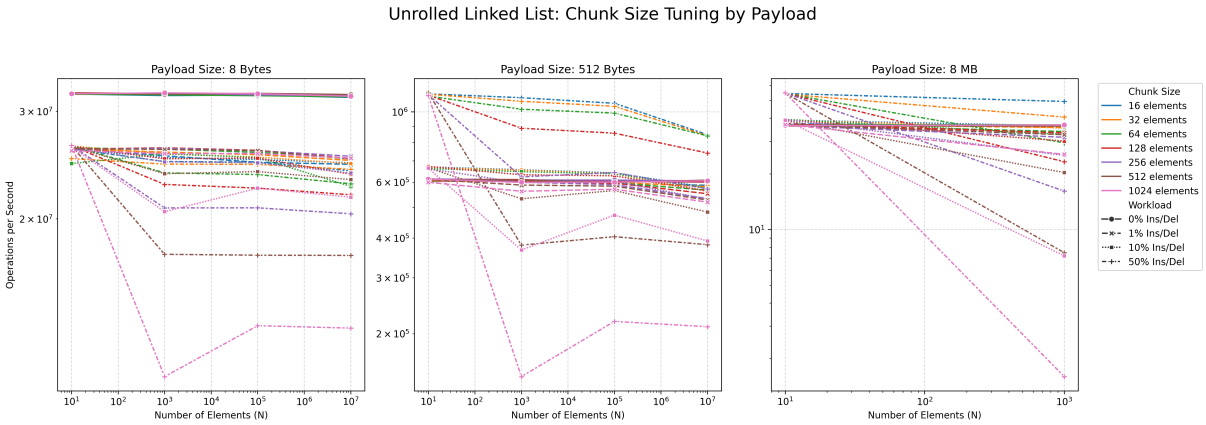
1 Unrolled Linked Lists **1**

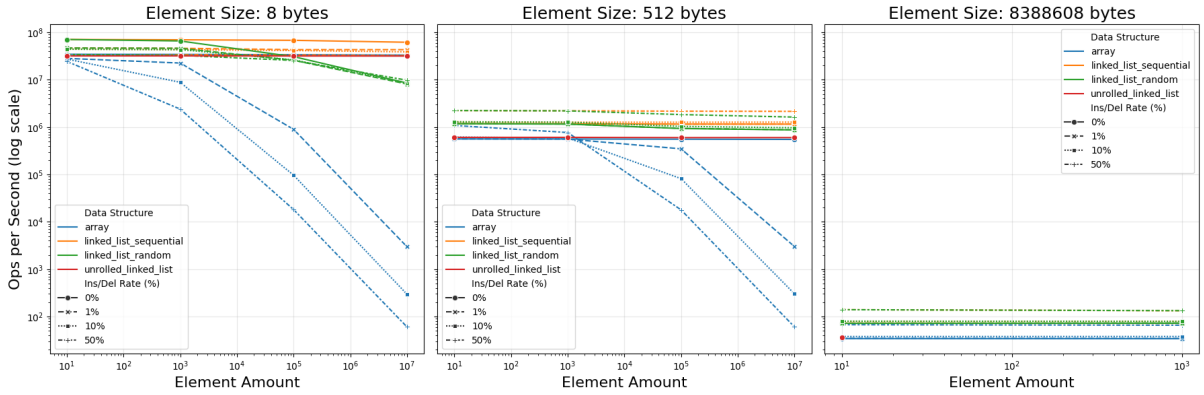
2 Tiered Arrays **2**

3 Extended Benchmarking **3**

1 Unrolled Linked Lists

In this assignment, the performance of contiguous arrays was compared against linked structures under varying workloads. The baseline Array was implemented using `std::vector<std::vector<char>>`. By structuring the array in this manner, the $O(N)$ penalty of copying massive payloads during heavy insertions is avoided, as only the small vector headers need to be shifted in memory. For the Unrolled Linked List, a `std::list<std::vector<char>>` was used. This structure provides the L1 cache locality of vectors, while allowing the linked list to handle chunk splitting when a chunk exceeds its capacity.



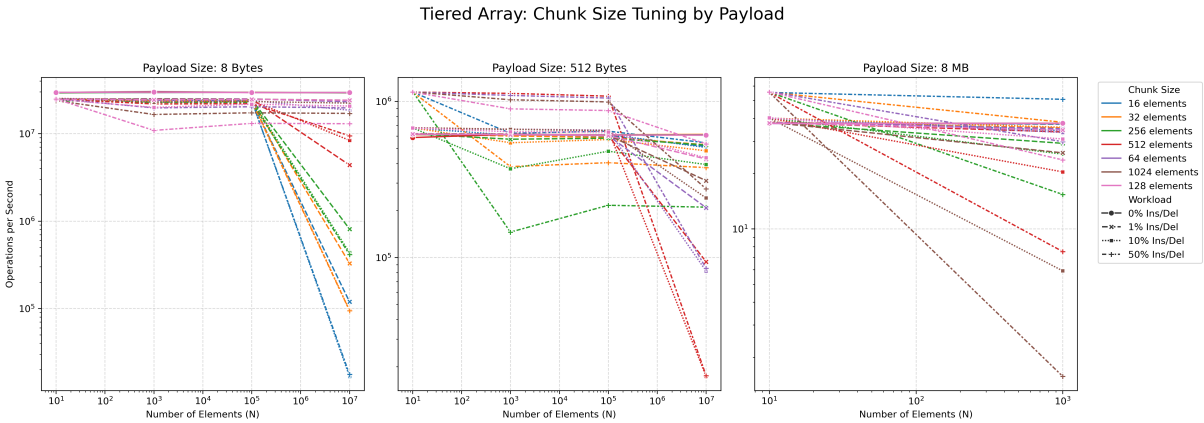


When the unrolled list was tuned across chunk sizes ranging from 16 to 1024 elements. At 8-byte payloads, finding the optimal chunk size was observed to be important, as the benchmark is primarily bottlenecked by structural pointer traversal. However, when scaled up to 8 MB elements, the performance of all tested data structures flatlined at roughly 100 operations per second. At this scale, the theoretical difference between $O(1)$ and $O(N)$ seems to become largely irrelevant, likely because the CPU is entirely bottlenecked by the physical memory bandwidth required to process the massive payloads.

An anomaly was consistently seen in the 512-byte tests: the 50% insert/delete workloads were recorded as faster than the 0% pure read workloads. Initially, this contradicts the assumption that pure traversal should be faster than shifting memory. Upon reviewing the test harness, it seems this might be caused by a differences in the computational workload. A read operation is forced to execute a pseudo-random number generator loop for every single byte, meaning a 512-byte read requires 512 separate math iterations. While, insertions and erasures rely on standard vector modifications. It is likely that these modifications utilize highly optimized, hardware-level SIMD instructions (such as memmove) to shift memory efficiently within the CPU cache. Shifting 512 bytes in hardware is most likely faster than calculating 512 individual math equations, suggesting that a heavy computational workload can mask the expected structural efficiencies of a data container.

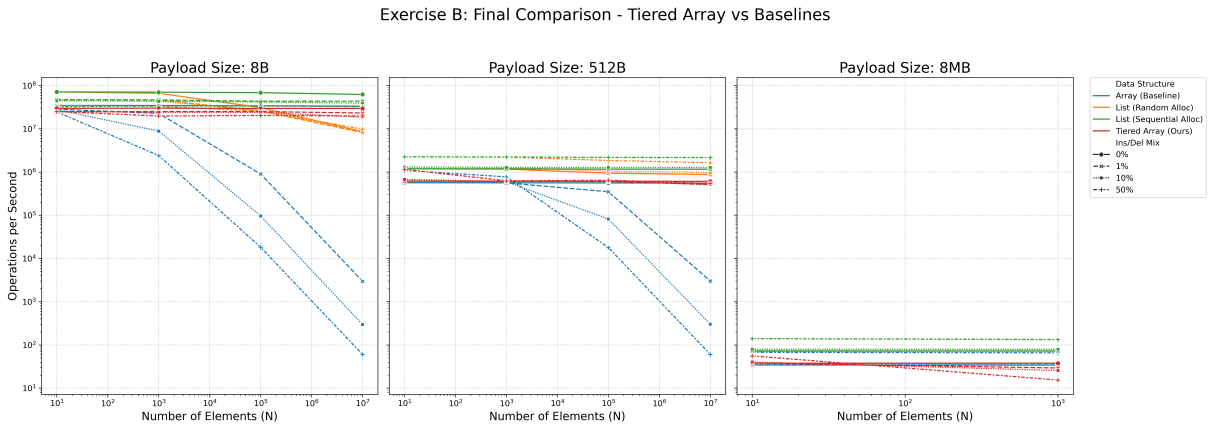
2 Tiered Arrays

The Tiered Array was implemented as a two-layer structure using a `std::vector` of `std::vector<char>` chunks. Similar to the baseline described in Section 1, this approach avoids large memory copies during insertions by limiting shifting operations to a single chunk. When a chunk exceeds its maximum capacity it is split into two halves and a new entry is added to the outer index vector.



The data structure was tuned using chunk sizes between 8 and 2048 elements to find the most efficient configuration. The results show that a chunk size of 256 elements offers the best performance balance.

Smaller sizes increase the overhead of the outer indirection layer while larger sizes lead to performance degradation during insertions as the chunks begin to behave like standard arrays.



In the final comparison the Tiered Array shows a performance trade-off. For pure sequential access it is slower than the sequentially allocated linked list because the two-layer indirection and chunk boundary checks introduce more overhead than simple pointer increments. However it significantly outperforms the randomly allocated linked list which represents a more realistic fragmented heap. Furthermore while the performance of the baseline Array collapses to near zero during 50% insertion workloads at large N the Tiered Array maintains a high and stable throughput. At the 8 MB payload scale the performance is again limited by memory bandwidth and all structures converge to the same result.

3 Extended Benchmarking